

Somali Police Force Case Management System

Core Project Blueprint - Part 1: System Overview, Tech Stack, & Core Schema

This document serves as Part 1 of the comprehensive system blueprint for the Somali Police Force Case Management System. It covers the executive system summary, architectural overview, tech stack, and the initial set of core database schemas.

Table of Contents

- Somali Police Force Case Management System
- Core Project Blueprint - Part 1: System Overview, Tech Stack, \& Core Schema
- Table of Contents
- 1. Executive Summary \& System Overview
 - 1.1. System Goals
 - 1.2. The 5-Tier Command Hierarchy
 - 1.3. User Persona and Role Matrices
- 2. Technical Architecture \& Core Tech Stack
 - 2.1. Backend Tech Stack
 - 2.2. Frontend Tech Stack
 - 2.3. Design System \& Theme Specification
- 3. Database Architecture (Part 1)
 - 3.1. Roles \& User Personnel Management
 - 3.1.1. Roles Table (`roles`)
 - 3.1.2. Users Table (`users`)
 - 3.1.3. Ranks Table (`ranks`)
 - 3.1.4. Police Officers Table (`police_officers`)
 - 3.2. Administrative Division Hierarchy Tables
 - 3.2.1. State Administrations Table (`state_administrations`)
 - 3.2.2. Regions Table (`regions`)
 - 3.2.3. Cities Table (`cities`)
 - 3.2.4. Districts Table (`districts`)
 - 3.3. Personnel Tracking \& Assignments
 - 3.3.1. Officer Assignments Table (`officer_assignments`)

- 3.3.2. Officer Transfers Table (`officer_transfers`)
-

1. Executive Summary & System Overview

1.1. System Goals

The Web-Based Case Management System (CMS) is designed for the Somali Police Force to provide a secure, digital pipeline for registering, investigating, tracking, and prosecuting criminal activities. The application digitizes manual Occurrence Book (OB) entries and structures them into official cases.

1.2. The 5-Tier Command Hierarchy

To match national administrative structures, the system implements a robust spatial scoping mechanism spanning:

1. **Federal/National Administration:** Oversees the entire system configuration.
2. **State Administration:** Commands police operations within a specific state.
3. **Regional Administration:** Oversees districts within a specified region.
4. **City/District Administration:** Coordinates station-level reports.
5. **Local Police Station:** Coordinates local patrols, OB entry, and initial case registration.

This structure enforces location-scoped data access, ensuring commanders at specific levels can only view records generated within their jurisdiction.

1.3. User Persona and Role Matrices

The system supports distinct user roles:

- **Admin:** Performs administrative controls, rank management, and core database backups.
 - **OB Staff (Occurrence Book):** Registers public complaints, inputs details into the digital ledger, and initiates drafts.
 - **Staff/Officer:** Investigates incidents and processes cases.
 - **Commanders:** Approve/reject cases, manage officer assignments, and generate station audits.
 - **CID (Criminal Investigation Dept):** Handles high-priority investigations and crime scene analyses.
 - **Prosecutors / Court Clerk / Judges:** Manage hearings, legal submissions, and issue court decisions.
 - **Jail Custodians:** Coordinate medical records, visitor logs, and release certificates.
-

2. Technical Architecture & Core Tech Stack

The application employs a decoupled client-server architecture.

2.1. Backend Tech Stack

- **Runtime:** Node.js
- **Framework:** Express.js
- **Database:** MySQL 8 (single data store — MongoDB config exists but is unused/optional, see Part 3B section 5 for details)
- **Connectivity:** mysql2/promise (connection pooling, async-await structure)
- **Authentication:** JSON Web Token (JWT) with secure cookies and bcrypt hashing (12 rounds)

2.2. Frontend Tech Stack

- **Framework:** Next.js 16 (React 19, Client & Server Component separation)
- **Build System:** Next dev (Turbopack)
- **UI Component Library:** Ant Design (AntD v6)
- **Styling Engine:** Vanilla CSS + TailwindCSS (utilities) + Next Google fonts (Inter)

2.3. Design System & Theme Specification

The system supports dual-theme profiles (Light Mode and Dark Mode) with the following attributes:

- **Transition Time:** 200ms ease on colors and backgrounds.
- **Theme Preference Persistence:** Automatically saved to `localStorage` and falls back to system preferences via media query on first visit.
- **Dark Mode Palette:**
- **Main Background:** `#0E0E0E`
- **Secondary/Sidebar/Header Background:** `#171717`
- **Card & Panel Elements:** `#1C1C1C`
- **Borders & Dividers:** `#2B2B2B`
- **Primary Text:** `#FFFFFF`
- **Secondary Text:** `#A5A5A5`
- **Muted/Placeholder Text:** `#707070`
- **Highlight Accent:** `#A8FF4D` (used selectively for selections, primary CTA hover states, and active tags)

3. Database Architecture (Part 1)

3.1. Roles & User Personnel Management

3.1.1. Roles Table (`roles`)

Defines permission tiers across the application.

```
CREATE TABLE IF NOT EXISTS roles (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL UNIQUE,
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3.1.2. Users Table (users)

Coordinates system authentication and user mappings.

```
CREATE TABLE IF NOT EXISTS users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  role_id INT NOT NULL,
  username VARCHAR(150) NOT NULL UNIQUE,
  email VARCHAR(150),
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(150),
  phone VARCHAR(30),
  `rank` VARCHAR(100),
  user_type ENUM('COMMANDER','OB_STAFF','STAFF') DEFAULT 'STAFF',
  assigned_level ENUM('ADMINISTRATION','STATE','REGION','DISTRICT_POLICE_STATION') DEFAULT NULL,
  state_administration_id INT,
  region_id INT,
  district_id INT,
  is_commander TINYINT(1) DEFAULT 0,
  status ENUM('ACTIVE','INACTIVE','SUSPENDED') DEFAULT 'ACTIVE',
  profile_image VARCHAR(500),
  is_active TINYINT(1) DEFAULT 1,
  last_login TIMESTAMP NULL,
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_users_role FOREIGN KEY (role_id) REFERENCES roles(id)
);
```

3.1.3. Ranks Table (ranks)

Details official police ranks.

```
CREATE TABLE IF NOT EXISTS ranks (
  id INT PRIMARY KEY AUTO_INCREMENT,
  rank_name VARCHAR(100) NOT NULL UNIQUE,
  rank_code VARCHAR(30) NOT NULL UNIQUE,
  description TEXT,
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

3.1.4. Police Officers Table (police_officers)

Stores official roster of police personnel.

```
CREATE TABLE IF NOT EXISTS police_officers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(150) NOT NULL,
  force_number VARCHAR(50) NOT NULL UNIQUE,
  rank_id INT,
```

```

phone VARCHAR(30),
email VARCHAR(150),
gender ENUM('male','female') DEFAULT 'male',
date_of_birth DATE,
address TEXT,
profile_image VARCHAR(500),
employment_status ENUM('active','suspended','retired','inactive') DEFAULT 'active',
created_by VARCHAR(100),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_officer_rank FOREIGN KEY (rank_id) REFERENCES ranks(id)
);

```

3.2. Administrative Division Hierarchy Tables

3.2.1. State Administrations Table (`state_administrations`)

```

CREATE TABLE IF NOT EXISTS state_administrations (
  id INT PRIMARY KEY AUTO_INCREMENT,
  state_name VARCHAR(150) NOT NULL,
  state_code VARCHAR(50) NOT NULL UNIQUE,
  username VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  commander_officer_id INT,
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_state_commander FOREIGN KEY (commander_officer_id) REFERENCES police_officers(id) ON DELETE
  SET NULL
);

```

3.2.2. Regions Table (`regions`)

```

CREATE TABLE IF NOT EXISTS regions (
  id INT PRIMARY KEY AUTO_INCREMENT,
  state_administration_id INT NOT NULL,
  region_name VARCHAR(150) NOT NULL,
  region_code VARCHAR(50) NOT NULL UNIQUE,
  username VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  commander_officer_id INT,
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_region_state FOREIGN KEY (state_administration_id) REFERENCES state_administrations(id) ON
  DELETE CASCADE,
  CONSTRAINT fk_region_commander FOREIGN KEY (commander_officer_id) REFERENCES police_officers(id) ON
  DELETE SET NULL
);

```

3.2.3. Cities Table (`cities`)

```

CREATE TABLE IF NOT EXISTS cities (
  id INT PRIMARY KEY AUTO_INCREMENT,
  region_id INT NOT NULL,
  city_name VARCHAR(150) NOT NULL,

```

```

city_code VARCHAR(50) NOT NULL UNIQUE,
username VARCHAR(150) NOT NULL UNIQUE,
password_hash VARCHAR(255) NOT NULL,
commander_officer_id INT,
created_by VARCHAR(100),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_city_region FOREIGN KEY (region_id) REFERENCES regions(id) ON DELETE CASCADE,
CONSTRAINT fk_city_commander FOREIGN KEY (commander_officer_id) REFERENCES police_officers(id) ON DELETE
SET NULL
);

```

3.2.4. Districts Table (districts)

```

CREATE TABLE IF NOT EXISTS districts (
  id INT PRIMARY KEY AUTO_INCREMENT,
  city_id INT NOT NULL,
  district_name VARCHAR(150) NOT NULL,
  district_code VARCHAR(50) NOT NULL UNIQUE,
  username VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  commander_officer_id INT,
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_district_city FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE CASCADE,
  CONSTRAINT fk_district_commander FOREIGN KEY (commander_officer_id) REFERENCES police_officers(id) ON
DELETE SET NULL
);

```

3.3. Personnel Tracking & Assignments

3.3.1. Officer Assignments Table (officer_assignments)

Tracks current physical status and placement of officers.

```

CREATE TABLE IF NOT EXISTS officer_assignments (
  id INT PRIMARY KEY AUTO_INCREMENT,
  officer_id INT NOT NULL,
  assignment_type VARCHAR(100) NOT NULL,
  assignment_id INT,
  is_current TINYINT(1) DEFAULT 1,
  assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  assigned_by VARCHAR(100),
  remarks TEXT,
  CONSTRAINT fk_assign_officer FOREIGN KEY (officer_id) REFERENCES police_officers(id) ON DELETE CASCADE
);

```

3.3.2. Officer Transfers Table (officer_transfers)

Logs historical transfers of officers between stations.

```

CREATE TABLE IF NOT EXISTS officer_transfers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  officer_id INT NOT NULL,
  from_assignment_type VARCHAR(100),

```

```
from_assignment_id INT,  
to_assignment_type VARCHAR(100),  
to_assignment_id INT,  
transfer_reason TEXT NOT NULL,  
transferred_by VARCHAR(100),  
transferred_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
remarks TEXT,  
CONSTRAINT fk_transfer_officer FOREIGN KEY (officer_id) REFERENCES police_officers(id) ON DELETE CASCADE  
);
```

Somali Police Force Case Management System

Core Project Blueprint - Part 2: Advanced Relational Database Schemas

This document serves as Part 2 of the comprehensive system blueprint for the Somali Police Force Case Management System, documenting advanced schemas for Incident/Case tracking, CID Investigations, Custody tracking, and Judicial Court integration.

3. Database Architecture (Part 2)

3.4. Cases & Occurrence Book (OB) Management

3.4.1. Occurrence Book Entries Table (`ob_entries`)

Stores initially reported public incidents recorded by OB staff at stations.

```
CREATE TABLE IF NOT EXISTS ob_entries (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  ob_number VARCHAR(50) NOT NULL UNIQUE,  
  incident_type VARCHAR(100) NOT NULL,  
  incident_location VARCHAR(255) NOT NULL,  
  description TEXT,  
  reported_by VARCHAR(150) NOT NULL,  
  reporter_phone VARCHAR(30),  
  registered_by_user_id INT NOT NULL,  
  registered_by_name VARCHAR(150) NOT NULL,  
  registered_by_role VARCHAR(100) NOT NULL,  
  registered_by_rank VARCHAR(100),  
  state_administration_id INT,  
  region_id INT,  
  district_id INT,  
  registration_date DATE NOT NULL,  
  registration_time TIME NOT NULL,  
  status ENUM('OB_REGISTERED', 'FORWARDED_FOR_REVIEW', 'CONVERTED_TO_CASE', 'CASE_OPENED', 'CLOSED') DEFAULT  
'OB_REGISTERED',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_ob_user FOREIGN KEY (registered_by_user_id) REFERENCES users(id),  
  CONSTRAINT fk_ob_state FOREIGN KEY (state_administration_id) REFERENCES state_administrations(id),  
  CONSTRAINT fk_ob_region FOREIGN KEY (region_id) REFERENCES regions(id),  
  CONSTRAINT fk_ob_district FOREIGN KEY (district_id) REFERENCES districts(id)  
);
```

3.4.2. Cases Table (`cases`)

Defines formalized criminal cases converted from OB entries.


```

CREATE TABLE IF NOT EXISTS cases (
  id INT PRIMARY KEY AUTO_INCREMENT,
  case_number VARCHAR(50) UNIQUE,
  case_title VARCHAR(255) NOT NULL,
  ob_number VARCHAR(30) NOT NULL UNIQUE,
  ob_entry_id INT,
  original_ob_staff_id INT,
  original_ob_staff_name VARCHAR(150),
  incident_type VARCHAR(100),
  complainant_name VARCHAR(150),
  complainant_phone VARCHAR(30),
  victim_name VARCHAR(150),
  description TEXT,
  incident_date DATETIME,
  incident_location VARCHAR(255),
  status VARCHAR(100) DEFAULT 'DRAFT',
  priority ENUM('low','medium','high','critical') DEFAULT 'medium',
  state_administration_id INT,
  region_id INT,
  city_id INT,
  district_id INT,
  assigned_officer_id INT,
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_case_state FOREIGN KEY (state_administration_id) REFERENCES state_administrations(id),
  CONSTRAINT fk_case_region FOREIGN KEY (region_id) REFERENCES regions(id),
  CONSTRAINT fk_case_city FOREIGN KEY (city_id) REFERENCES cities(id),
  CONSTRAINT fk_case_district FOREIGN KEY (district_id) REFERENCES districts(id),
  CONSTRAINT fk_case_officer FOREIGN KEY (assigned_officer_id) REFERENCES police_officers(id)
);

```

3.4.3. Case Confirmations Table (`case_confirmations`)

Logs district and regional commander approvals.

```

CREATE TABLE IF NOT EXISTS case_confirmations (
  id INT PRIMARY KEY AUTO_INCREMENT,
  case_id INT NOT NULL,
  commander_user_id VARCHAR(100) NOT NULL,
  confirmation_status VARCHAR(50),
  comments TEXT,
  confirmed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_conf_case FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE
);

```

3.4.4. Blockchain Verification Records Table (`blockchain_records`)

Maintains SHA-256 chain integrity verification to prevent backend tampering.

```

CREATE TABLE IF NOT EXISTS blockchain_records (
  id INT PRIMARY KEY AUTO_INCREMENT,
  entity_type VARCHAR(50) NOT NULL,
  entity_id INT NOT NULL,
  proof_version INT NOT NULL DEFAULT 1,
  sha256_hash VARCHAR(64) NOT NULL,
  previous_hash VARCHAR(64),
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

3.4.5. Case Transfers Table (case_transfers)

```
CREATE TABLE IF NOT EXISTS case_transfers (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  case_id INT NOT NULL,  
  from_state_administration_id INT,  
  from_region_id INT,  
  from_city_id INT,  
  from_district_id INT,  
  to_state_administration_id INT,  
  to_region_id INT,  
  to_city_id INT,  
  to_district_id INT,  
  from_officer_id INT,  
  to_officer_id INT,  
  transferred_by VARCHAR(100),  
  transfer_reason TEXT NOT NULL,  
  transfer_type VARCHAR(50) NOT NULL,  
  transferred_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  blockchain_record_id INT,  
  CONSTRAINT fk_ctrans_case FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE,  
  CONSTRAINT fk_ctrans_bc FOREIGN KEY (blockchain_record_id) REFERENCES blockchain_records(id)  
);
```

3.5. Criminal Profiling, Witnesses, & Custody

3.5.1. Complainants Table (complainants)

```
CREATE TABLE IF NOT EXISTS complainants (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  full_name VARCHAR(150) NOT NULL,  
  gender ENUM('male','female') DEFAULT 'male',  
  age INT,  
  nationality VARCHAR(100) DEFAULT 'Somali',  
  id_type VARCHAR(50),  
  id_number VARCHAR(100),  
  phone VARCHAR(30),  
  address TEXT,  
  email VARCHAR(150),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3.5.2. Criminals Profile & Biometrics Table (criminals)

```
CREATE TABLE IF NOT EXISTS criminals (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  full_name VARCHAR(150) NOT NULL,  
  mother_name VARCHAR(150),  
  alias VARCHAR(150),  
  gender ENUM('male','female') DEFAULT 'male',  
  date_of_birth DATE,  
  age INT,  
  nationality VARCHAR(100) DEFAULT 'Somali',  
  id_type VARCHAR(50),  
  id_number VARCHAR(100),  
  phone VARCHAR(30),  
  address TEXT,  
  description TEXT,
```

```

photo_url VARCHAR(500),
offender_photo VARCHAR(500),
face_capture_image VARCHAR(500),
face_capture_notes TEXT,
profile_notes TEXT,
arrest_status ENUM('not_arrested','arrested','released','wanted') DEFAULT 'not_arrested',
fingerprint_hash VARCHAR(255),
biometric_notes TEXT,
is_arrested TINYINT(1) DEFAULT 0,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

3.5.3. Case Criminal Mappings Table (case_criminals)

Stores many-to-many relationship of suspects linked to cases.

```

CREATE TABLE IF NOT EXISTS case_criminals (
  id INT PRIMARY KEY AUTO_INCREMENT,
  case_id INT NOT NULL,
  criminal_id INT NOT NULL,
  linked_by_user_id VARCHAR(100),
  linked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status ENUM('active','removed') DEFAULT 'active',
  role_in_case VARCHAR(150),
  notes TEXT,
  added_by VARCHAR(100),
  added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_cc_case FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE,
  CONSTRAINT fk_cc_criminal FOREIGN KEY (criminal_id) REFERENCES criminals(id),
  UNIQUE KEY uq_case_criminal (case_id, criminal_id)
);

```

3.5.4. Evidence Ledger Table (evidence)

Tracks collected physical, video, audio, or document submissions.

```

CREATE TABLE IF NOT EXISTS evidence (
  id INT PRIMARY KEY AUTO_INCREMENT,
  case_id INT NOT NULL,
  evidence_number VARCHAR(50) NOT NULL UNIQUE,
  type ENUM('document','photo','video','audio','physical','other') DEFAULT 'document',
  title VARCHAR(255) NOT NULL,
  description TEXT,
  file_url VARCHAR(500),
  file_size BIGINT,
  mime_type VARCHAR(100),
  collected_by VARCHAR(100),
  collection_date DATE,
  location_found VARCHAR(255),
  status ENUM('collected','analyzed','submitted','returned') DEFAULT 'collected',
  hash_sha256 VARCHAR(64),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_ev_case FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE
);

```

3.5.5. Chain of Custody Table (chain_of_custody)

```
CREATE TABLE IF NOT EXISTS chain_of_custody (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  evidence_id INT NOT NULL,  
  transferred_from VARCHAR(100),  
  transferred_to VARCHAR(100) NOT NULL,  
  transfer_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  reason TEXT,  
  location VARCHAR(255),  
  notes TEXT,  
  CONSTRAINT fk_coc_evidence FOREIGN KEY (evidence_id) REFERENCES evidence(id) ON DELETE CASCADE  
);
```

3.5.6. Arrest & Sentence Status Table (arrests)

```
CREATE TABLE IF NOT EXISTS arrests (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  case_id INT NOT NULL,  
  suspect_id INT NOT NULL,  
  police_station_id INT,  
  arrested_by VARCHAR(100),  
  arrest_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  arrest_location VARCHAR(255),  
  charges TEXT,  
  court_decision ENUM('pending', 'convicted', 'acquitted', 'dismissed', 'adjourned') DEFAULT 'pending',  
  court_decision_notes TEXT,  
  sentence_period_value INT,  
  sentence_period_unit ENUM('days', 'months', 'years'),  
  sentence_start_date DATE,  
  expected_release_date DATE,  
  actual_release_date DATE,  
  sentence_status  
  ENUM('awaiting_trial', 'sentenced', 'serving', 'release_review', 'completed', 'released', 'wanted', 'escaped', 'acquitted', 'dismissed') DEFAULT 'awaiting_trial',  
  final_status VARCHAR(100),  
  bail_status ENUM('no_bail', 'bail_granted', 'bail_pending') DEFAULT 'no_bail',  
  bail_amount DECIMAL(12,2),  
  notes TEXT,  
  CONSTRAINT fk_ar_case FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE,  
  CONSTRAINT fk_ar_suspect FOREIGN KEY (suspect_id) REFERENCES criminals(id) ON DELETE CASCADE,  
  CONSTRAINT fk_ar_station FOREIGN KEY (police_station_id) REFERENCES districts(id) ON DELETE SET NULL  
);
```

3.6. CID (Criminal Investigation Department)

3.6.1. CID Assigned Cases Table (cid_cases)

```
CREATE TABLE IF NOT EXISTS cid_cases (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  police_case_id INT NOT NULL UNIQUE,  
  case_number VARCHAR(50) NOT NULL,  
  ob_number VARCHAR(50),  
  case_title VARCHAR(255),  
  crime_category VARCHAR(150),  
  priority ENUM('low', 'medium', 'high', 'critical') DEFAULT 'medium',  
  assigned_officer VARCHAR(150),  
  assigned_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

supervisor VARCHAR(150),
assignment_status ENUM('assigned','accepted','reassigned','rejected') DEFAULT 'assigned',
investigation_status
ENUM('open','under_investigation','evidence_collection','witness_interviews','suspect_tracking','arrest_made',
'investigation_completed','supervisor_review','approved','rejected','sent_to_prosecutor','sent_to_court'
) DEFAULT 'open',
investigation_started_at TIMESTAMP NULL,
findings TEXT,
recommendations TEXT,
supervisor_notes TEXT,
prosecutor_forwarded_at TIMESTAMP NULL,
created_by VARCHAR(100),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_cid_case_police FOREIGN KEY (police_case_id) REFERENCES cases(id) ON DELETE CASCADE
);

```

3.6.2. CID Crime Scenes Visited Table (cid_crime_scenes)

```

CREATE TABLE IF NOT EXISTS cid_crime_scenes (
id INT PRIMARY KEY AUTO_INCREMENT,
cid_case_id INT NOT NULL,
location VARCHAR(255) NOT NULL,
date_visited DATE,
officer VARCHAR(150),
observations TEXT,
scene_photos TEXT,
collected_evidence TEXT,
created_by VARCHAR(100),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_cid_scene_case FOREIGN KEY (cid_case_id) REFERENCES cid_cases(id) ON DELETE CASCADE
);

```

3.7. Prisoner Custody & Releases

3.7.1. Release Approvals Ledger Table (release_approvals)

Triggers multiple validation nodes (Admin, Prison warden, and Court review) before generating the final release certificate.

```

CREATE TABLE IF NOT EXISTS release_approvals (
id INT PRIMARY KEY AUTO_INCREMENT,
suspect_id INT NOT NULL,
arrest_id INT NOT NULL,
requested_by VARCHAR(100),
requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
request_reason TEXT NOT NULL,
status
ENUM('pending','approved','pending_admin_review','admin_reviewed','prison_confirmed','court_approved','certificate_generated','released','rejected') DEFAULT 'pending_admin_review',
admin_reviewed_by VARCHAR(100),
admin_reviewed_at TIMESTAMP NULL,
admin_review_notes TEXT,
prison_confirmed_by VARCHAR(100),
prison_confirmed_at TIMESTAMP NULL,
prison_confirmation_notes TEXT,
court_approved_by VARCHAR(100),
court_approved_at TIMESTAMP NULL,

```

```

court_approval_notes TEXT,
certificate_number VARCHAR(80),
certificate_issued_by VARCHAR(100),
certificate_issued_at TIMESTAMP NULL,
certificate_notes TEXT,
reviewed_by VARCHAR(100),
reviewed_at TIMESTAMP NULL,
review_notes TEXT,
CONSTRAINT fk_rel_suspect FOREIGN KEY (suspect_id) REFERENCES criminals(id) ON DELETE CASCADE,
CONSTRAINT fk_rel_arrest FOREIGN KEY (arrest_id) REFERENCES arrests(id) ON DELETE CASCADE
);

```

3.8. Legal & Judicial Court Mappings

3.8.1. Court Cases Table (`court_cases`)

Coordinates police transfer tracking into court-specific registers.

```

CREATE TABLE IF NOT EXISTS court_cases (
  id INT PRIMARY KEY AUTO_INCREMENT,
  court_case_number VARCHAR(50) NOT NULL UNIQUE,
  police_case_id INT NOT NULL,
  police_case_number VARCHAR(50),
  ob_number VARCHAR(50),
  case_title VARCHAR(255),
  crime_category VARCHAR(100),
  case_description TEXT,
  source_status VARCHAR(50),
  status ENUM('registered', 'awaiting_hearing', 'hearing_scheduled', 'in_trial', 'judgment_issued',
'sentenced', 'appealed', 'closed', 'archived') DEFAULT 'registered',
  created_by VARCHAR(100),
  registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  assigned_judge VARCHAR(150),
  assigned_prosecutor VARCHAR(150),
  final_outcome ENUM('convicted', 'acquitted', 'dismissed'),
  closure_reason TEXT,
  closure_date DATE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_court_cases_police FOREIGN KEY (police_case_id) REFERENCES cases(id) ON DELETE CASCADE
);

```

3.8.2. Court Hearings Scheduled Table (`court_hearings`)

```

CREATE TABLE IF NOT EXISTS court_hearings (
  id INT PRIMARY KEY AUTO_INCREMENT,
  court_case_id INT NOT NULL,
  hearing_type VARCHAR(100) NOT NULL,
  hearing_date DATE NOT NULL,
  hearing_time TIME NOT NULL,
  court_room VARCHAR(50),
  assigned_judge VARCHAR(150),
  status ENUM('scheduled', 'completed', 'cancelled') DEFAULT 'scheduled',
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_hearings_case FOREIGN KEY (court_case_id) REFERENCES court_cases(id) ON DELETE CASCADE
);

```

Somali Police Force Case Management System

Core Project Blueprint - Part 3: Frontend Architecture, Routes, Layouts, & Theme Context

This document serves as Part 3 of the comprehensive system blueprint for the Somali Police Force Case Management System, detailing page routing structures, Context API state pipelines, layouts, and custom CSS theme tokens.

4. Frontend Application Architecture

4.1. Router Mapping & Dynamic Directories

The React frontend uses Next.js App Router for nested directory structures:

- `src/app/login/page.jsx` : Handles user authentication inputs.
- `src/app/dashboard/` : Contains dashboard variations loaded by user role:
- `/admin/page.jsx` : Admin KPIs, user activation tables, stations counts.
- `/cid/page.jsx` : CID investigator queues, investigation tracks, reports.
- `/court/page.jsx` : Court hearings tracking list, outcomes, sentences.
- `/jail/page.jsx` : Jail registrations list, visitor logs, prisoner releases.
- `/unit/page.jsx` : Performance charts, unit-level stats.
- `src/app/cases/` : Case grids, filters, and new case registration layouts.
- `src/app/offenders/page.jsx` : Lists criminal records and facial capture items.
- `src/app/stations/page.jsx` : Details police stations and hierarchies.
- `src/app/police-officers/page.jsx` : Roster of police officers.
- `src/app/reports/page.jsx` : Statistics, audits, and reports overview.

4.1.1. Offender/Suspect Photo Rendering Rule

- Offender/suspect records with no photo (`photo_url` , `offender_photo` , or `face_capture_image` all `NULL`) must render a generic fallback avatar (icon or initials-based) in every view — Offenders list, suspect cards, and case details. No stock or downloaded images are ever used as photo placeholders.
-

4.2. Context Mappings (Context API)

4.2.1. Authentication Context (AuthContext.js)

Coordinates login tokens, payload parsing, roles assignments, and local user caching.

```
// Located in /frontend/src/contexts/AuthContext.js
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  ...
};
export const useAuth = () => useContext(AuthContext);
```

4.2.2. Theme Context (ThemeContext.js)

Manages system theme states, sets root element data attributes, and supports persistence via local storage.

```
// Located in /frontend/src/contexts/ThemeContext.js
export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState('light');
  ...
};
export const useTheme = () => useContext(ThemeContext);
```

4.3. Base Layout & Sub-Navigation

4.3.1. Main Layout Shell (AppLayout.jsx)

Exposes sidebar and top navbar components, wrapping the core canvas inside `<ConfigProvider>` to toggle Ant Design custom tokens:

```
// Located in /frontend/src/components/layout/AppLayout.jsx
<ConfigProvider theme={theme === 'dark' ? darkTheme : lightTheme}>
  <Layout style={{ minHeight: '100vh' }}>
    <Sidebar collapsed={collapsed} />
    <Layout>
      <TopNavbar collapsed={collapsed} setCollapsed={setCollapsed} />
      <Content className="app-content">{children}</Content>
    </Layout>
  </Layout>
</ConfigProvider>
```

4.3.2. Sidebar Component (Sidebar.jsx)

Determines role accessibility arrays, shows local profile badges, and filters navigation menus dynamically based on current user roles.

4.3.3. Top Navbar (TopNavbar.jsx)

Holds header actions: notifications bell dropdown panel, profile context menus, and the new dark mode toggle switcher.

4.4. Design Tokens & Styling Customizations

4.4.1. Ant Design Custom Tokens Configuration (theme.js)

Forms light and dark theme configurations mapping primary palettes to Ant Design's `<ConfigProvider>` context:

- **Dark Mode Custom Config:**

```
export const darkTheme = {
  algorithm: theme.darkAlgorithm,
  token: {
    colorPrimary: '#A8FF4D',      // Accent Lime Highlight
    colorBgBase: '#0E0E0E',      // Core Main Background
    colorBgLayout: '#171717',    // Layout Background
    colorBgContainer: '#1C1C1C', // Card & Panel Background
    colorBorder: '#2B2B2B',      // Border Lines
    colorText: '#FFFFFF',        // Primary Text
    colorTextSecondary: '#A5A5A5',
    colorTextPlaceholder: '#707070',
  }
};
```

4.4.2. High-Specificity Global Styling Overrides (globals.css)

Appends dark mode selectors under `[data-theme='dark']` to resolve CSS specificity leaks:

- **Transitions:** Smooth 200ms ease animations applied globally:

```
body, .app-content, .ant-card, .ant-table, .ant-input, .ant-btn {
  transition: background 200ms ease, border-color 200ms ease, color 200ms ease !important;
}
```

- **Specific Overrides:**
- **Card & Panel:** Target `.standard-panel` and `.standard-metric-card` to render `#1C1C1C` background and `#2B2B2B` borders with `!important`.
- **Hero Container:** Styles `.standard-dashboard-hero` background to a dark gradient with `#171717` and `#0E0E0E` colors.
- **Top Navbar controls:** Sets `.topbar-toggle` and `.topbar-icon-button` background to `#1C1C1C` and borders to `#2B2B2B`.
- **Custom Side Lists:** Mapped `.standard-side-list>div` backgrounds to `#171717` and borders to `#2B2B2B`.
- **Regular Buttons:** Override `.ant-btn:not(.ant-btn-primary)` with `#171717` background and `#2B2B2B` borders.
- **Accents:** Configured `.standard-metric-green` and hover borders to `#A8FF4D` highlights.

Somali Police Force Case Management System

Core Project Blueprint - Part 3B: Database Schema Verification & Live Delta

This document lists the actual tables currently present in the running database, detailing schemas for tables missing from the initial blueprint, and cataloging any differences or deprecated definitions.

1. Complete List of Current Live Tables (47 tables)

The following tables are currently active in the live MySQL `police_cms` database:

1. `arrests`
2. `audit_logs`
3. `biometric_identifiers`
4. `blockchain_records`
5. `case_actions`
6. `case_confirmations`
7. `case_criminals`
8. `case_transfers`
9. `case_victims`
10. `cases`
11. `chain_of_custody`
12. `cid_cases`
13. `cid_crime_scenes`
14. `cid_progress_notes`
15. `cid_reports`
16. `cities`
17. `complainants`
18. `court_appeals`
19. `court_cases`
20. `court_evidence_notes`
21. `court_hearings`
22. `court_judgments`
23. `court_proceedings`
24. `court_sentences`

25. court_witnesses
26. criminals
27. districts
28. evidence
29. login_logs
30. ob_entries
31. officer_assignments
32. officer_transfers
33. police_officers
34. prison_transfers
35. prisoner_documents
36. prisoner_medical_records
37. prisoner_visitor_logs
38. ranks
39. referrals
40. regions
41. release_approvals
42. roles
43. state_administrations
44. users
45. victims
46. witness_statements
47. witnesses

2. Live Column Structures for Missing/Undocumented Tables

The following schemas detail the actual column structure (types, constraints, keys) for tables that exist in the live database but were omitted from the Part 1 & Part 2 blueprints:

Table: audit_logs

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	varchar(100)	YES	''	NULL	''
user_email	varchar(150)	YES	''	NULL	''
action	varchar(100)	NO	''	NULL	''
entity_type	varchar(100)	YES	''	NULL	''

Field	Type	Null	Key	Default	Extra
entity_id	int	YES	''	NULL	''
details	json	YES	''	NULL	''
old_data	json	YES	''	NULL	''
new_data	json	YES	''	NULL	''
ip_address	varchar(50)	YES	''	NULL	''
user_agent	text	YES	''	NULL	''
created_at	timestamp	YES	MUL	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: biometric_identifiers

Field	Type	Null	Key	Default	Extra
id	int	NO	PR I	NULL	auto_increment
suspect_id	int	NO	MU L	NULL	''
biometric_type	enum('fingerprint','face','iris','other')	NO	MU L	NULL	''
biometric_hash	varchar(255)	NO	''	NULL	''
quality_score	decimal(5,2)	YES	''	NULL	''
captured_by	varchar(100)	YES	''	NULL	''
captured_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED
notes	text	YES	''	NULL	''

Table: case_actions

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
case_id	int	NO	MUL	NULL	''
performed_by	varchar(100)	YES	''	NULL	''

Field	Type	Null	Key	Default	Extra
action_type	varchar(100)	NO	''	NULL	''
description	text	YES	''	NULL	''
status_before	varchar(100)	YES	''	NULL	''
status_after	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: case_victims

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
case_id	int	NO	MUL	NULL	''
victim_id	int	NO	MUL	NULL	''
notes	text	YES	''	NULL	''
added_by	varchar(100)	YES	''	NULL	''
added_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: cid_progress_notes

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
cid_case_id	int	NO	MUL	NULL	''
note	text	NO	''	NULL	''
status	varchar(100)	YES	''	NULL	''
created_by	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: cid_reports

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
cid_case_id	int	NO	MUL	NULL	''

Field	Type	Null	Key	Default	Extra
report_title	varchar(255)	NO	''	NULL	''
case_summary	text	YES	''	NULL	''
activities	text	YES	''	NULL	''
evidence_summary	text	YES	''	NULL	''
witness_summary	text	YES	''	NULL	''
suspect_analysis	text	YES	''	NULL	''
findings	text	NO	''	NULL	''
recommendations	text	YES	''	NULL	''
submitted_by	varchar(100)	YES	''	NULL	''
submitted_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: court_appeals

Field	Type	Null	Key	Default	Extra
id	int	NO	PRIMARY	NULL	auto_increment
court_case_id	int	NO	MULTI	NULL	''
filed_by	varchar(150)	NO	''	NULL	''
appeal_reason	text	NO	''	NULL	''
filing_date	date	NO	''	NULL	''
status	enum('pending', 'approved', 'rejected')	YES	''	pending	''
created_by	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: court_evidence_notes

Field	Type	Null	Key	Default	Extra
court_case_id	int	NO	PRI	NULL	''

Field	Type	Null	Key	Default	Extra
evidence_id	int	NO	PRI	NULL	''
notes	text	YES	''	NULL	''

Table: court_judgments

Field	Type	Null	Key	Default	Extra
id	int	NO	PR I	NULL	auto_increment
court_case_id	int	NO	MU L	NULL	''
judge_name	varchar(150)	YES	''	NULL	''
decision_date	date	NO	''	NULL	''
decision_type	enum('convicted', 'acquitted', 'dismissed')	NO	''	NULL	''
judgment_summary	text	NO	''	NULL	''
created_by	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: court_proceedings

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
court_case_id	int	NO	MUL	NULL	''
hearing_id	int	NO	MUL	NULL	''
proceeding_date	date	NO	''	NULL	''
notes	text	YES	''	NULL	''
judge_remarks	text	YES	''	NULL	''
prosecutor_remarks	text	YES	''	NULL	''
defense_remarks	text	YES	''	NULL	''

Field	Type	Null	Key	Default	Extra
created_by	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: court_sentences

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
court_case_id	int	NO	MUL	NULL	''
defendant_name	varchar(150)	NO	''	NULL	''
sentence_type	varchar(100)	NO	''	NULL	''
duration	varchar(100)	YES	''	NULL	''
fine_amount	decimal(15,2)	YES	''	NULL	''
sentence_date	date	NO	''	NULL	''
created_by	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: court_witnesses

Field	Type	Null	Key	Default	Extra
id	int	NO	PR I	NULL	auto_increment
court_case_id	int	NO	MU L	NULL	''
witness_id	int	NO	MU L	NULL	''
status	enum('summoned','present','absent')	YES	''	summoned	''
testimony	text	YES	''	NULL	''
signed_statement_url	varchar(500)	YES	''	NULL	''
summoned_at	timestamp	YES	''	NULL	''

Field	Type	Null	Key	Default	Extra
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: login_logs

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	varchar(100)	YES	''	NULL	''
username	varchar(150)	YES	''	NULL	''
success	tinyint(1)	NO	''	0	''
failure_reason	varchar(255)	YES	''	NULL	''
ip_address	varchar(50)	YES	''	NULL	''
user_agent	text	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: prison_transfers

Field	Type	Null	Key	Default	Extra
id	int	NO	PR I	NULL	auto_increment
suspect_id	int	NO	MU L	NULL	''
arrest_id	int	YES	MU L	NULL	''
from_facility	varchar(255)	YES	''	NULL	''
to_facility	varchar(255)	NO	''	NULL	''
transfer_reason	text	NO	''	NULL	''
transfer_date	datetime	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED
authorized_by	varchar(100)	YES	''	NULL	''

Field	Type	Null	Key	Default	Extra
status	enum('pending','completed','cancelled')	YES	''	completed	''
notes	text	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: prisoner_documents

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
suspect_id	int	NO	MUL	NULL	''
arrest_id	int	YES	MUL	NULL	''
document_type	varchar(100)	NO	''	NULL	''
title	varchar(255)	NO	''	NULL	''
file_url	varchar(500)	YES	''	NULL	''
file_hash	varchar(64)	YES	''	NULL	''
uploaded_by	varchar(100)	YES	''	NULL	''
uploaded_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED
notes	text	YES	''	NULL	''

Table: prisoner_medical_records

Field	Type	Null	Key	Default	Extra
id	int	NO	PR I	NULL	auto_increment
suspect_id	int	NO	MU L	NULL	''
arrest_id	int	YES	MU L	NULL	''
record_date	datetime	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Field	Type	Null	Key	Default	Extra
condition_summary	text	NO	''	NULL	''
treatment_given	text	YES	''	NULL	''
doctor_name	varchar(150)	YES	''	NULL	''
facility	varchar(255)	YES	''	NULL	''
fitness_status	enum('fit','needs_treatment','hospitalized','critical')	YES	''	fit	''
recorded_by	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: prisoner_visitor_logs

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
suspect_id	int	NO	MUL	NULL	''
arrest_id	int	YES	MUL	NULL	''
visitor_name	varchar(150)	NO	''	NULL	''
visitor_id_number	varchar(100)	YES	''	NULL	''
relationship	varchar(100)	YES	''	NULL	''
visit_date	datetime	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED
purpose	text	YES	''	NULL	''
approved_by	varchar(100)	YES	''	NULL	''
notes	text	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: referrals

Field	Type	Null	Key	Default	Extra
id	int	NO	PRIMARY	NULL	auto_increment
case_id	int	NO	MULTI	NULL	''
referred_by	varchar(100)	YES	''	NULL	''
referred_to_role	varchar(50)	YES	''	NULL	''
referred_to_user	varchar(100)	YES	''	NULL	''
reason	text	YES	''	NULL	''
notes	text	YES	''	NULL	''
status	enum('pending','accepted','rejected','completed')	YES	''	pending	''
response	text	YES	''	NULL	''
referred_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED
responded_at	timestamp	YES	''	NULL	''

Table: victims

Field	Type	Null	Key	Default	Extra
id	int	NO	PRIMARY	NULL	auto_increment
full_name	varchar(150)	NO	''	NULL	''
gender	enum('male','female')	YES	''	male	''
age	int	YES	''	NULL	''
nationality	varchar(100)	YES	''	Somali	''
id_type	varchar(50)	YES	''	NULL	''
id_number	varchar(100)	YES	''	NULL	''

Field	Type	Null	Key	Default	Extra
phone	varchar(30)	YES	''	NULL	''
address	text	YES	''	NULL	''
injury_description	text	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: witness_statements

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
case_id	int	NO	MUL	NULL	''
witness_id	int	NO	MUL	NULL	''
statement	text	NO	''	NULL	''
statement_date	date	YES	''	NULL	''
taken_by	varchar(100)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

Table: witnesses

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
full_name	varchar(150)	NO	''	NULL	''
gender	enum('male', 'female')	YES	''	male	''
age	int	YES	''	NULL	''
phone	varchar(30)	YES	''	NULL	''
address	text	YES	''	NULL	''
relationship_to_case	varchar(150)	YES	''	NULL	''
created_at	timestamp	YES	''	CURRENT_TIMESTAMP	DEFAULT_GENERATED

3. Schema Differences in Documented Tables

Comparing the live column definitions for the original documented tables:

- **Status:** Verified all live table definitions. No structural column discrepancies or type-mismatches exist between the live DB and the documented tables. All columns mapped correctly.
-

4. Deprecated/Unused Tables

The following tables were previously documented but do not exist in the live database or are unused:

- **None:** All tables documented in Part 1 and Part 2 are active in the live MySQL database.
-

5. Secondary Data Store: MongoDB Integration

Based on direct inspection of the codebase:

1. Connection Mappings:

- **File Path:** The connection helper is configured in `mongodb.js`.
- **Library:** The app uses the `mongoose` (v9.6.2) Object Data Modeling (ODM) library.
- **Connection String:** Resolved via `process.env.MONGODB_URI` from `.env`.
- **Initialization:** The connection function `connectMongoDB()` is loaded on startup inside `app.js` (line 99).

1. Current Usage Status:

- **Skipped on Startup:** In `.env`, `MONGODB_URI` is left blank, meaning `connectMongoDB()` outputs `MongoDB connection skipped: MONGODB_URI is not set` on startup and returns `null`.
- **No DB Operations:** Codebase searches confirm `mongoose` is never imported, and MongoDB collections or schemas are never defined outside the config file. All state actions (including audit logs and session logs) write directly to the MySQL database.
- **Functional Requirement:** MongoDB is completely **optional** and **unused** in the current system implementation. All core operations continue to function relying solely on MySQL.

Somali Police Force Case Management System

Core Project Blueprint - Part 4: API Documentation

This document serves as Part 4 of the comprehensive system blueprint for the Somali Police Force Case Management System, detailing REST API endpoints, routing patterns, controller mappings, middleware chains, and required roles.

4. REST API Endpoint Catalog

All routes are prefixed by the base URL: `http://localhost:5000/api`

4.1. Authentication Module (`/api/auth`)

Handles system authentication, profile fetching, and session destruction.

Method	Path	Controller Method	Middleware	Auth Required	Roles Allowed	Description
POST	<code>/login</code>	<code>login</code>	None	No	Any	Logs in a user, returning a JWT token inside cookies or payload.
POST	<code>/logout</code>	<code>logout</code>	<code>authMiddleware</code>	Yes	Any	Invalidates the user session.
GET	<code>/me</code>	<code>getMe</code>	<code>authMiddleware</code>	Yes	Any	Returns profile details of the logged-in user.

Example Request/Response:

- **POST `/api/auth/login`:**
- **Request Body:**

```
{
  "username": "admin",
  "password": "password123"
}
```

- **Success Response (200 OK):**

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
```

```

    "username": "admin",
    "role": "admin",
    "full_name": "System Administrator"
  }
}

```

- **Failure Response (401 Unauthorized):**

```

{
  "success": false,
  "message": "Invalid username or password"
}

```

4.2. Occurrence Book Module (/api/ob-entries)

Manages initial public complaint entries at local stations.

Method	Path	Controller Method	Middlewares	Auth Required	Roles Allowed	Description
GET	/	getObEntries	authMiddleware, allowRoles	Yes	admin, ob_staff, staff, officer, district_admin, cid, cid_director, cid_supervisor, cid_officer, STATE_COMMANDER, REGION_COMMANDER, DISTRICT_COMMANDER, POLICE_STATION_COMMANDER	Retrieves all OB entries scoped to the user's location jurisdiction.
GET	/:id	getObEntryById	authMiddleware, allowRoles	Yes	Same as GET / roles	Fetches details of a specific OB entry by ID.
POST	/	createObEntry	authMiddleware, allowRoles	Yes	admin, ob_staff, officer, district_admin	Creates a new OB entry in the system.
POST	/:id/convert-to-case	convertObToCase	authMiddleware, allowRoles	Yes	admin, officer, district_admin, cid, cid_director, cid_supervisor, cid_officer, STATE_COMMANDER, REGION_COMMANDER, DISTRICT_COMMANDER, POLICE_STATION_COMMANDER	Converted specified OB entry into an active case.

4.3. Case Management Module (/api/cases)

Handles the lifecycle of formal investigations.

Method	Path	Controller Method	Middleware	Auth Required	Roles Allowed	Description
GET	/	getCases	authMiddleware, allowRoles	Yes	admin, staff, officer, district_admin, cid, cid_director, cid_supervisor, cid_officer, STATE_COMMANDER, REGION_COMMANDER, DISTRICT_COMMANDER, POLICE_STATION_COMMANDER	Lists location-scoped cases.
GET	/stats	getCasesStats	authMiddleware, allowRoles	Yes	Same as GET / roles + officer	Retrieves count matrices of cases (open, pending, closed, critical).
GET	/my-assigned	getMyAssignedCases	authMiddleware, allowRoles	Yes	Same as GET / roles + officer	Lists cases assigned to the authenticated officer.
GET	/assignable/officers	getAssignableOfficers	authMiddleware, allowRoles	Yes	admin, district_commander, police_station_commander, district_admin	Fetches officers eligible for case assignment.
GET	/:id	getCasesById	authMiddleware, allowRoles	Yes	Same as GET / roles	Details a case and links to witnesses, victims, and suspects.
GET	/:id/export	exportCasePackage	authMiddleware, allowRoles	Yes	Same as GET / roles	Exports all case evidence and statements into a compressed PDF/ZIP package.
POST	/	createCase	authMiddleware, allowRoles	Yes	admin, officer, district_admin, cid, cid_director, cid_supervisor, cid_officer	Creates a new case draft.
POST	/:id/court-decision	recordCourtDecision	authMiddleware, allowRoles	Yes	admin, court	Records judicial trial outcomes.
PATCH	/:id/assign	assignCaseOfficer	authMiddleware, allowRoles	Yes	admin, district_commander, police_station_commander, district_admin	Assigns an officer to a specific case.
PUT	/:id	updateCase	authMiddleware, allowRoles	Yes	admin, officer, district_admin, cid, cid_director, cid_supervisor, cid_officer	Updates case status or updates data points.

4.4. CID Investigations Module (/api/cid)

Manages advanced CID case escalations, supervisors' reviews, and crime scene registers.

Method	Path	Controller Method	Middleware	Auth Required	Roles Allowed	Description
GET	/dashboard	getCidDashboard	authMiddleware, allowRoles	Yes	admin, cid, cid_director, cid_supervisor, cid_officer, prosecutor_liaison	Fetches CID-specific metrics.
GET	/cases	getCidCases	authMiddleware, allowRoles	Yes	Same as GET /dashboard	Lists CID investigations.
POST	/cases/sync	syncCidCase	authMiddleware, allowRoles	Yes	Same as GET /dashboard + STATE_COMMANDER, REGION_COMMANDER, DISTRICT_COMMANDER, POLICE_STATION_COMMANDER	Synchronizes station-level cases to the CID database.
GET	/cases/:id	getCidCaseById	authMiddleware, allowRoles	Yes	Same as GET /dashboard	Fetches details of a single CID investigation.
PATCH	/cases/:id/assign	assignCidCase	authMiddleware, allowRoles	Yes	admin, cid, cid_director, cid_supervisor, prosecutor_liaison, ...COMMANDER_ROLES	Assigns a CID case to an investigator.
PATCH	/cases/:id/acknowledge	acknowledgeCidCase	authMiddleware, allowRoles	Yes	Same as GET /dashboard	Acknowledges receipt of case by the assigned investigator.
PATCH	/cases/:id/investigation	updateInvestigation	authMiddleware, allowRoles	Yes	admin, cid, cid_director, cid_supervisor, cid_officer	Appends status notes and logs search findings.
POST	/cases/:id/crime-scenes	addCrimeScene	authMiddleware, allowRoles	Yes	Same as PATCH /:id/investigation	Logs physical evidence found at the crime scene.
POST	/cases/:id/reports	submitReport	authMiddleware, allowRoles	Yes	Same as PATCH /:id/investigation	Submits final investigation reports.

Method	Path	Controller Method	Middleware	Auth Required	Roles Allowed	Description
PATCH	/cases/:id/review	reviewInvestigation	authMiddleware, allowRoles	Yes	admin, cid, cid_director, cid_supervisor, prosecutor_liaison, ...COMMANDER_ROLES	Submits supervisor review notes.
POST	/cases/:id/forward-prosecutor	forwardToProsecutor	authMiddleware, allowRoles	Yes	Same as PATCH /:id/review	Forwards evidence package to the prosecution office.

4.5. Prisoner Custody Module (/api/custody)

Coordinates medical logs, booking notes, visitors, and releases.

Method	Path	Controller Method	Middleware	Auth Required	Roles Allowed	Description
GET	/prisoners	getPrisoners	authMiddleware, allowRoles	Yes	admin, jail, court, judge, prosecutor	Lists all booked prisoners.
GET	/prisoners/:id	getPrisonerById	authMiddleware, allowRoles	Yes	Same as GET /prisoners	Retrieves a prisoner's full folder, including warrants.
POST	/prisoners	bookPrisoner	authMiddleware, allowRoles	Yes	admin, jail	Performs prisoner intake booking.
POST	/prisoners/:id/medical	addMedicalRecord	authMiddleware, allowRoles	Yes	admin, jail	Logs physical checks and hospital referrals.
POST	/prisoners/:id/visitors	logVisitor	authMiddleware, allowRoles	Yes	admin, jail	Logs visitor details.
POST	/prisoners/:id/transfers	transferPrisoner	authMiddleware, allowRoles	Yes	admin, jail	Logs facility transfers.
GET	/releases	getReleaseRequests	authMiddleware, allowRoles	Yes	Same as GET /prisoners	Lists pending release approvals.
POST	/releases/request	requestRelease	authMiddleware, allowRoles	Yes	admin, jail	Submits a request for release.
PATCH	/releases/:id/approve	approveRelease	authMiddleware, allowRoles	Yes	admin, court, jail	Validates a step in the release approval pipeline.

4.6. Judicial Court Module (`/api/court`)

Coordinates scheduling hearings, court actions, and sentencing entries.

Method	Path	Description
GET	<code>/api/court/cases</code>	Lists all court-referred cases
GET	<code>/api/court/cases/:id</code>	Court case detail
GET	<code>/api/court/dashboard</code>	Court dashboard metrics/status counts
GET	<code>/api/court/notifications</code>	Court notifications
GET	<code>/api/court/calendar</code>	Hearings calendar (supports <i>daterange</i> , <i>courtroom</i> , <i>case_status</i> filters)
GET	<code>/api/court/personnel</code>	Court personnel list (judges etc.)
POST	<code>/api/court/cases</code>	Files a case with the court
POST	<code>/api/court/cases/:id/hearings</code>	Schedules a hearing for a case
POST	<code>/api/court/hearings/:hearingId/proceedings</code>	Logs proceedings/judge remarks for a hearing
POST	<code>/api/court/cases/:id/judgments</code>	Records the judgment
POST	<code>/api/court/cases/:id/sentences</code>	Records the sentence
POST	<code>/api/court/cases/:id/appeals</code>	Files an appeal
PATCH	<code>/api/court/cases/:id/assign</code>	Assigns court personnel to a case
PATCH	<code>/api/court/cases/:id/close</code>	Closes a court case

4.7. General System Modules

- **Ranks** (`/api/ranks`): GET `/`, POST `/`, DELETE `/:id` (Admin only)
- **Police Officers** (`/api/police-officers`): GET `/`, POST `/`, PATCH `/:id`, DELETE `/:id` (Admin/Commanders)
- **Blockchain** (`/api/blockchain`): GET `/verify/:caseId` (Calculates blockchain hash changes to detect modifications)
- **Search** (`/api/search`): GET `/` (Global search by case number, name, or ob number)
- **Notifications** (`/api/notifications`): GET `/`, PATCH `/:id/read` (Fetches system warnings and notification logs)

Somali Police Force Case Management System

Core Project Blueprint - Part 5: Authentication & Authorization

This document serves as Part 5 of the comprehensive system blueprint for the Somali Police Force Case Management System, detailing authentication mechanisms, roles structures, JWT specifications, and routing permissions validation gates.

5. Security & Access Control Infrastructure

5.1. The Unified Authentication Flow

The system processes login requests using a unified authentication algorithm that aggregates credentials checks across different databases tables:

```
sequenceDiagram
    autonumber
    actor Client as User Client
    participant Auth as Auth Controller
    participant DB as MySQL Database
    participant JWT as JWT Service

    Client->>Auth: POST /api/auth/login (username/email, password)
    Auth->>DB: Query username/email across UNION (users, state_administrations, regions, cities, districts)
    alt User Not Found
        DB-->>Auth: Empty rows
        Auth->>DB: Insert failure log (login_logs)
        Auth-->>Client: 401 Unauthorized (Invalid credentials)
    else User Found
        DB-->>Auth: Return id, username, email, role, password_hash, is_active, location details
        alt Account Deactivated
            Auth->>DB: Insert failure log (login_logs)
            Auth-->>Client: 403 Forbidden (Account is deactivated)
        else Account Active
            Auth->>Auth: bcrypt.compare(password, password_hash)
            alt Password Mis-match
                Auth->>DB: Insert failure log (login_logs)
                Auth-->>Client: 401 Unauthorized (Invalid credentials)
            else Password Matches
                Auth->>DB: Update last_login timestamp (for regular users)
                Auth->>DB: Insert successful log (login_logs)
                Auth->>JWT: sign(payload, JWT_SECRET, { expiresIn: '24h' })
                JWT-->>Auth: Encoded token
                Auth-->>Client: 200 OK (payload + token)
            end
        end
    end
end
```

5.2. Password Hashing Specification

- **Library:** `bcryptjs`
 - **Workflow:** On user creation or password updates, passwords undergo 12 salt rounds hashing.
 - **Storage Column:** `password_hash` (`VARCHAR(255)`)
-

5.3. JWT & Session Implementation

Authentication is token-based. Clients must attach the returned JWT token to every request inside the `Authorization` header prefixed with `Bearer` .

Token Payload Schema:

```
{
  "id": 1,
  "username": "admin",
  "email": "admin@spf.gov.so",
  "role": "admin",
  "roleCode": "admin",
  "fullName": "System Admin",
  "phone": "+252611111111",
  "rank": "General",
  "userType": "COMMANDER",
  "assignedLevel": "ADMINISTRATION",
  "isCommander": true,
  "scopeType": null,
  "scopeId": null,
  "location": {
    "stateId": 1,
    "stateName": "Banadir",
    "regionId": null,
    "regionName": null,
    "districtId": null,
    "districtName": null
  },
  "iat": 1721245000,
  "exp": 1721331400
}
```

- **Expiration Time:** Fixed at `24h` (configurable in `JWT_EXPIRES_IN` env).
-

5.4. Middleware Gates

1. Authentication Gate (`authMiddleware.js`):

- Extracts token from `Authorization` header.
- Runs `jwt.verify(token, process.env.JWT_SECRET)` .
- Sets `req.user = decoded` on success.
- Catches `TokenExpiredError` (returns `401 Token has expired`) or invalid signatures (returns `401 Invalid token`).

1. Authorization Role Gate (`roleMiddleware.js`):

- Evaluates permissions using `allowRoles(...roles)` factory.
- Matches `req.user.role` against list. On failure, responds with `403 Access Denied`.

5.5. Permission Matrix Per Role

Role	OB Entries (Read/Write)	Cases (Read/Write)	Assignments/Transfers	CID Investigations	Custody & Bookings	Judicial Hearings
admin	Write / Read	Write / Read	Write / Read	Write / Read	Write / Read	Write / Read
ob_staff	Write / Read	Read	No	No	No	No
officer	Write / Read	Write / Read	Read	Read	Read	Read
state_commander	Read	Read	Write / Read	Read	Read	Read
region_commander	Read	Read	Write / Read	Read	Read	Read
district_commander	Read	Read	Write / Read	Read	Read	Read
policestationcommander	Read	Read	Write / Read	Read	Read	Read
state_admin	Read	Read	No	No	No	No
region_admin	Read	Read	No	No	No	No
city_admin	Read	Read	No	No	No	No
district_admin	Write / Read	Write / Read	Write / Read	Read	Read	Read
cid_director	Read	Write / Read	Read	Write / Read	Read	Read
cid_supervisor	Read	Write / Read	Read	Write / Read	Read	Read
cid_officer	Read	Write / Read	Read	Write / Read	Read	Read
prosecutor	Read	Read	No	Read	Read	Write / Read
judge	Read	Read	No	Read	Read	Write / Read
court_clerk	Read	Read	No	No	Read	Write / Read
jail	Read	Read	No	No	Write / Read	No

Somali Police Force Case Management System

Core Project Blueprint - Part 6: Business Modules Specification

This document serves as Part 6 of the comprehensive system blueprint for the Somali Police Force Case Management System, detailing business objectives, logic engines, status transitions, validation parameters, and relationships between the four core operations modules.

6. Functional Module Specification

```
graph TD
  OB[Occurrence Book OB] -->|Commander Review & Approval| Case[Case Management]
  Case -->|Referral to CID| CID[CID Investigation]
  Case -->|Charge & Referral| Court[Court Trial Proceeding]
  Court -->|Sentence Serving/Warrant| Jail[Prisoner Custody]
  Jail -->|Validation Pipeline| Release[Release Approval System]
```

6.1. Module 1: Occurrence Book & Case Management

6.1.1. Purpose

The entry point of information. Digitizes local complaints, converts them into formal cases, allows commanders to assign police personnel, and guides investigations from drafts to judicial escalation.

6.1.2. Main Implementation Files

- **Controller:** backend/src/controllers/caseController.js , obEntryController.js
- **Router:** backend/src/routes/caseRoutes.js , obEntryRoutes.js
- **Services:** backend/src/services/cidService.js , courtService.js

6.1.3. Database Mappings

- Primary tables: ob_entries , cases , case_confirmations , case_actions , case_transfers .
- Ancillary tables: complainants , criminals , case_criminals , victims , case_victims , witnesses , witness_statements , evidence .

6.1.4. Key Business & Transition Rules

- **Occurrence Book (OB) Entry Conversion:**
- Status starts at OB_REGISTERED .

- Converting an OB entry creates a matching entry in the `cases` table and updates `ob_entries.status` to `CONVERTED_TO_CASE`.
 - Converting an OB entry to a case automatically marks the case as referred to CID (creates the `cid_cases` row via `ensureCidCaseForPoliceCase`) at the same time it is set to `under_investigation` — there is no separate manual "refer to CID" step anymore.
 - **Case Status Validation:**
 - Status checks follow `CASE_STATUS_FLOW`:
 - `draft` → `registered` → `referred_to_cid` → `under_investigation` → `ready_for_court` → `forwarded_to_court` → `court_decided` → `closed`
 - Allowed Transitions:
 - `registered` → `referred_to_cid` | `under_investigation`
 - `referred_to_cid` → `under_investigation`
 - `under_investigation` → `ready_for_court` | `forwarded_to_court` | `referred_to_cid`
 - The assigned CID officer must still acknowledge the case before updating investigation notes — that gate is unchanged.
 - **Date validation:** Incident time must be at least one hour in the past (`validateIncidentDate`).
-

6.2. Module 2: CID (Criminal Investigation Department)

6.2.1. Purpose

Manages investigations of high-priority felonies, forensic tracking of crime scenes, supervisor reviews, and preparing legal dossiers for prosecution liaison.

6.2.2. Main Implementation Files

- **Controller:** `backend/src/controllers/cidController.js`
- **Router:** `backend/src/routes/cidRoutes.js`
- **Services:** `backend/src/services/cidService.js`

6.2.3. Database Mappings

- Primary tables: `cid_cases`, `cid_progress_notes`, `cid_crime_scenes`, `cid_reports`.
- Shared tables: `cases`, `evidence`, `case_criminals`, `arrests`.

6.2.4. Key Business Rules

- **Auto-Synchronization:** Accessing the CID dashboard invokes `syncAllCidCases`, ensuring any case flagged as `referred_to_cid` at the local level is automatically synchronized to the `cid_cases` workspace.
- **Investigation Lifecycles:**
- Default status: `open`.
- Transitions: `open` → `under_investigation` → `evidence_collection` → `witness_interviews` → `suspect_tracking` → `arrest_made` → `investigation_completed` → `supervisor_review` -

> `approved` -> `sent_to_prosecutor` / `sent_to_court`.

- **Acknowledge Check:** The assigned CID officer must acknowledge receipt of a case before updating investigation notes.

6.3. Module 3: Prisoner Custody Management

6.3.1. Purpose

Coordinates prisoner bookings, facility transfers, visitor logs, medical histories, and the strict multi-signoff prisoner release validation chain.

6.3.2. Main Implementation Files

- **Controller:** `backend/src/controllers/custodyController.js`
- **Router:** `backend/src/routes/custodyRoutes.js`

6.3.3. Database Mappings

- Primary tables: `biometric_identifiers`, `prisoner_documents`, `prison_transfers`, `prisoner_medical_records`, `prisoner_visitor_logs`, `release_approvals`.
- Core reference tables: `criminals` (suspect identifier), `arrests` (sentence identifier), `districts` (station reference).

6.3.4. Key Business Rules

- **Biometrics Mappings:**
- Recording a suspect's fingerprint or face biometrics automatically updates the global `criminals` index column `fingerprint_hash`.
- **Release Approval Validation Pipeline:**
- To release a prisoner, a release approval workflow is initialized.
- Release approval status transition flow:
 1. `pending_admin_review`: Triggered by a jail custodian.
 2. `admin_reviewed`: Checked and approved by the administrator.
 3. `court_approved`: Checked and confirmed by the presiding judge/court.
 4. `prison_confirmed`: Confirmed by the prison supervisor.
 5. `certificate_generated`: The system generates a certificate number: `REL - [YEAR] - [APPROVAL_ID_PADDED]`.
 6. `released`: Prisoner record is closed.
- **Arrest Status Derivation:**
- `criminals.is_arrested` and `criminals.arrest_status` are derived from the criminal's most recent/active `arrests.sentence_status` record, and must always be kept in sync automatically whenever an arrest is recorded or updated — never edited independently:

```
| arrests.sentence_status | is_arrested | arrest_status | |---|---|---| | awaiting_trial,  
sentenced, serving, release_review | 1 | arrested | | wanted, escaped | 1 | wanted | | released,  
completed, acquitted, dismissed | 0 | released | | (no arrest record) | 0 | not_arrested |
```

6.4. Module 4: Legal & Judicial Court Module

6.4.1. Purpose

Tracks court prosecutions, hearings schedules, testimonies, trial proceedings, and sentences.

6.4.2. Main Implementation Files

- **Controller:** `backend/src/controllers/courtController.js`
- **Router:** `backend/src/routes/courtRoutes.js`
- **Services:** `backend/src/services/courtService.js`

6.4.3. Database Mappings

- Primary tables: `court_cases`, `court_hearings`, `court_witnesses`, `court_evidence_notes`, `court_proceedings`, `court_judgments`, `court_sentences`, `court_appeals`.

6.4.4. Key Business Rules

- **Referred Intake:**
- When a case status changes to `forwarded_to_court` in local case management, `ensureCourtCaseForPoliceCase` creates a court docket under status `registered`.
- **Trial status changes:**
- Transitions: `registered` -> `awaiting_hearing` -> `hearing_scheduled` -> `in_trial` -> `judgment_issued` -> `sentenced` -> `closed`.
- **Outcomes Linking:**
- Recording a `court_judgment` updates `court_cases.final_outcome` to `convicted`, `acquitted`, or `dismissed`.
- If a suspect is convicted, the judge submits a sentence mapping duration, fines, and jail assignments, linking back to the prisoner custody booking.

Somali Police Force Case Management System

Core Project Blueprint - Part 7: Error Handling, Security, Logging, & Performance

This document serves as Part 7 of the comprehensive system blueprint for the Somali Police Force Case Management System, detailing global error middlewares, database constraint parsers, input sanitations, blockchain audits, and database query optimizations.

7. Security, Error Management, Logging, & Performance

7.1. Global Error Handling Strategy

The Express application utilizes a centralized error-handling middleware (`errorHandler.js`) that captures all errors passed via `next(err)` across routes.

Database Exception Mapping:

- **Duplicate Keys (`ER_DUP_ENTRY`)**: Returns a `409 Conflict` status with a message stating that the record already exists.
 - **Foreign Key Violations (`ER_NO_REFERENCED_ROW_2`)**: Returns a `400 Bad Request` status indicating that the referenced parent resource (e.g. station ID, user ID) does not exist.
 - **Payload Too Large (Status 413)**: Returns a `413 Payload Too Large` status (typically triggered when file uploads exceed the 5MB body size limit).
 - **Stack Traces**: Enforces env checks to only append detailed stack traces in `development` mode, hiding them in production to prevent source code leaks.
-

7.2. Application & Network Security

7.2.1. SQL Injection Protection

- The application enforces parameterization on all database operations:
- Raw queries utilize `?` place-holders combined with argument arrays parsed by the driver:

```
await db.query('SELECT * FROM cases WHERE id = ?', [caseId]);
```

- Enforces typed variables casting on values like limits and offsets (`parseInt(limit)`) before inserting them into query strings to prevent string-based injection.

7.2.2. XSS (Cross-Site Scripting) & CORS

- **Body Size Restrictions:** Restricts JSON/url-encoded body payloads to a maximum of `10MB`.
- **CORS Block:** Integrated the `cors()` middleware wrapper in the API pipeline to regulate resource accesses by authorized hostnames.
- **File Uploads Sandbox:** Uploaded evidence files and profile photos are placed inside `/uploads` and served as static resources, preventing remote execution of scripts.

7.2.3. Cryptography & Key Storage

- **Secret Storage:** Key parameters (JWT secrets, DB credentials, ports) are decoupled into an environment file (`.env`).
 - **Bcrypt:** Direct password storage is prohibited. All passwords undergo 12 salt rounds hashing.
-

7.3. Audit & Logging Engines

7.3.1. Database Audit Logs (`audit_logs`)

Every status change, profile update, or resource creation calls `writeAuditLog()`. It logs:

- User identifiers (`user_id`, `user_email`).
- Action tags (e.g., `'CREATE_CASE'`, `'LOGIN'`, `'LOGOUT'`, `'UPDATE_STATUS'`).
- Targeted entity schema information (`entity_type`, `entity_id`).
- Before-and-after data snapshots (`old_data`, `new_data`) stored as JSON.
- Machine context details (`ip_address`, `user_agent`).

7.3.2. Authentication Auditing (`login_logs`)

Dedicated table tracking authentication status logs:

- Records all successful and failed logins.
 - Captures failure reasons (e.g., `'Invalid credentials'`, `'Account is deactivated'`).
-

7.4. Performance & Query Optimization

7.4.1. Database Indexes

Core indexes are placed on columns frequently queried by filters and joins:

- `idx_users_location` on `users(state_administration_id, region_id, district_id)` to optimize location scoping checks.
- `idx_users_type` on `users(user_type, assigned_level)`.
- `idx_case_ob_entry` on `cases(ob_entry_id)`.
- `idx_case_original_ob_staff` on `cases(original_ob_staff_id)`.
- `idx_ob_location` on `ob_entries(state_administration_id, region_id, district_id)`.

- `idx_audit_logs_created_at` on `audit_logs(created_at)`.

7.4.2. Pagination Mappings

Retrieval lists (such as `/api/cases`, `/api/ob-entries`, `/api/suspects`) enforce pagination using `LIMIT ? OFFSET ?` clauses to prevent buffer overflow. Returns metadata objects detailing count statistics:

```
"pagination": {  
  "page": 1,  
  "limit": 20,  
  "total": 142,  
  "pages": 8  
}
```

Somali Police Force Case Management System

Core Project Blueprint - Part 8: Deployment, Code Statistics, & Appendix

This document serves as Part 8 of the comprehensive system blueprint for the Somali Police Force Case Management System, detailing configuration structures, production deployment guides, file statistics, and glossary indices.

8. Deployment, System Statistics, & Glossary

8.1. Local & Production Deployment Guides

8.1.1. System Prerequisites

- **Runtime:** Node.js ($\geq$ v18.0)
- **DBMS:** MySQL Server ($\geq$ 8.0)
- **MongoDB:** Running instance (for syncing logs or background caches)

8.1.2. Local Development Setup

1. Repository Setup:

Clone the repository and locate both the `backend` and `frontend` folders.

1. Environment Variables:

- Create a `.env` file in the `backend` folder:

```
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=yourpassword
DB_NAME=police_cms
JWT_SECRET=yourjwtsecretkey
JWT_EXPIRES_IN=24h
MONGODB_URI=mongodb://localhost:27017/police_cms_logs # Optional (Not required for the app to run)
```

- Create a `.env` file in the `frontend` folder:

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
```

1. Database Initialization:

- Open MySQL client and run the database initialization schema:

```
mysql -u root -p < backend/database/schema.sql
```

- Seed the initial administrative structures, roles, and ranks:

```
node backend/database/seed.js
```

1. Installing Dependencies:

- Inside the `/backend` folder: `npm install`
- Inside the `/frontend` folder: `npm install`

1. Running Locally:

- Backend server: `node src/app.js` (starts on port 5000)
- Frontend next app: `npm run dev` (starts on port 3000)

8.1.3. Production Build & Deployment

- **Backend PM2 Process Management:**

Deploy using a process manager like PM2 to guarantee automatic restarts:

```
npm install -g pm2
pm2 start src/app.js --name "spf-case-system-backend"
```

- **Next.js Production Build:**

Compile Next.js client-side assets to optimized production build folder:

```
npm run build
npm run start
```

8.2. Source Code Statistics

Metric	Measured Value
Combined Source Files (<code>/backend/src</code> & <code>/frontend/src</code>)	119 files
Combined Source Folders	48 directories
REST Router files	28 route modules
REST Controller files	28 controller modules
Database Schema Tables (<code>schema.sql</code>)	37 tables

Metric	Measured Value
TODO/FIXME comments	None found

8.3. Module Dependency Graph

```
graph TD
  Client[Next.js App Router] -->|Axios REST Calls| API[Express API Server]
  API -->|JWT verification| Auth[Authentication Middleware]
  API -->|Scope parameters verification| Scope[Location Scoping Helper]
  API -->|Audit Logger| Audit[Audit Logs DB table]
  API -->|SQL queries| MySQL[(MySQL Database)]
  API -->|Sync| Mongo[(MongoDB logs backup)]
```

8.4. Appendix & Glossary

8.4.1. Terms & Definitions

- **OB (Occurrence Book)**: The primary logbook at a police station where all initial complaints are logged.
- **CID**: Criminal Investigation Department. Responsible for complex felony cases.
- **5-Tier Scope**: State Administration -> Region -> City -> District -> Local Station.
- **Scoping**: Data visibility bounds checked dynamically using the user's location IDs.

8.4.2. Code Naming Conventions

- **Database Columns**: snake_case (e.g. `case_number`, `assigned_officer_id`).
- **Express Routes & Controllers**: camelCase (e.g. `getObEntries`, `createCase`).
- **React Components**: PascalCase (e.g. `TopNavbar.jsx`, `StandardDashboard.jsx`).

8.4.3. Shared System Utilities

- `writeAuditLog`: Saves actions to the `audit_logs` table.
- `generateCaseNumber`: Creates unique incrementing codes formatted like `CASE-YYYY-XXXXXX`.
- `generateOBNumber`: Creates unique codes formatted like `OB-YYYY-XXXXXX`.